PRAKTIKUM SISTEM TERDISTRIBUSI

Ditulis oleh

Galet Guntoro Setiaji, S.Kom., M.Kom

FAKULTAS TEKNOLOGI KOMUNIKASI DAN INFORMATIKA

PROGRAM STUDI TEKNIK INFORMATIKA

UNIVERSITAS SEMARANG

Gambaran Project Akhir

Mahasiswa akan membuat sistem sederhana, tentang “Sistem Manajemen Produk & Order berbasis Microservices”.

Terdiri dari:

- Service 1: User Service
- Service 2: Product Service
- Service 3: Order Service
- API Gateway sederhana
- Komunikasi antar service via HTTP (REST API)

Kebutuhan Sistem

1. Hardware Minimum

   RAM: 4 GB (disarankan 8 GB)

   Storage: 10 GB kosong

   Processor: Dual Core

2. Sistem Operasi

   Windows 10/11 / Linux / MacOS

3. Software Pendukung
   - Python 3.10+
   - pip
   - Postman / Insomnia (testing API)
   - VS Code + Extension Python
   - Git (opsional)
4. Library Python

   Install : *pip install fastapi uvicorn requests*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> pip install fastapi uvicorn requests
Collecting fastapi
  Downloading fastapi-0.135.2-py3-none-any.whl (117 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 117.4/117.4 kB 1.7 MB/s eta 0:00:00
Collecting uvicorn
  Downloading uvicorn-0.42.0-py3-none-any.whl (68 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 68.8/68.8 kB ? eta 0:00:00
Collecting requests
  Downloading requests-2.33.0-py3-none-any.whl (65 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 65.0/65.0 kB ? eta 0:00:00
Collecting typing-extensions>=4.8.0
  Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Collecting starlette>=0.46.0
  Downloading starlette-1.0.0-py3-none-any.whl (72 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 72.7/72.7 kB ? eta 0:00:00
Collecting annotated-doc>=0.0.2
  Downloading annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Collecting typing-inspection>=0.4.2
  Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Collecting pydantic>=2.9.0
  Downloading pydantic-2.12.5-py3-none-any.whl (463 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 463.6/463.6 kB 4.8 MB/s eta 0:00:00
Collecting click>=7.0
  Downloading click-8.3.1-py3-none-any.whl (108 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 108.3/108.3 kB 6.1 MB/s eta 0:00:00
```

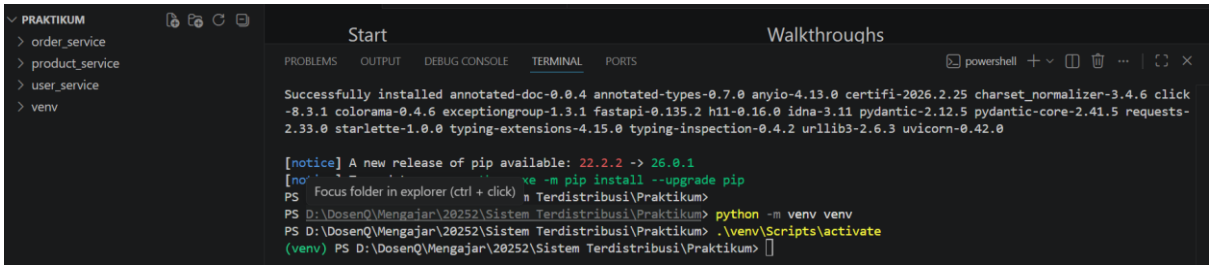
```
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 71.0/71.0 kB 3.8 MB/s eta 0:00:00
Collecting certifi>=2023.5.7
  Downloading certifi-2026.2.25-py3-none-any.whl (153 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 153.7/153.7 kB 4.5 MB/s eta 0:00:00
Collecting colorama
  Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting annotated-types>=0.6.0
  Downloading annotated_types-0.7.0-py3-none-any.whl (13 kB)
Collecting pydantic-core==2.41.5
  Downloading pydantic_core-2.41.5-cp310-cp310-win_amd64.whl (2.0 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 7.5 MB/s eta 0:00:00
Collecting anyio<5,>=3.6.2
  Downloading anyio-4.13.0-py3-none-any.whl (114 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 114.4/114.4 kB 6.5 MB/s eta 0:00:00
Collecting exceptiongroup>=1.0.2
  Downloading exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Installing collected packages: urllib3, typing-extensions, idna, h11, colorama, charset_normalizer, certifi, annotated-types, annotated
-doc, typing-inspection, requests, pydantic-core, exceptiongroup, click, uvicorn, pydantic, anyio, starlette, fastapi
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.13.0 certifi-2026.2.25 charset_normalizer-3.4.6 click-8.3.1 co
lorama-0.4.6 exceptiongroup-1.3.1 fastapi-0.135.2 h11-0.16.0 idna-3.11 pydantic-2.12.5 pydantic-core-2.41.5 requests-2.33.0 starlette-1
.0.0 typing-extensions-4.15.0 typing-inspection-0.4.2 urllib3-2.6.3 uvicorn-0.42.0

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum>
```

Pertemuan 1 – Pengenalan Microservices & Setup

1. Struktur Folder Project

Root Folder **PRAKTIKUM**, didalan ada sub folder dengan nama folder sebagai berikut.

- *user_service*
- *product_service*
- *order_service*



2. Kemudian Setup Virtual Environment

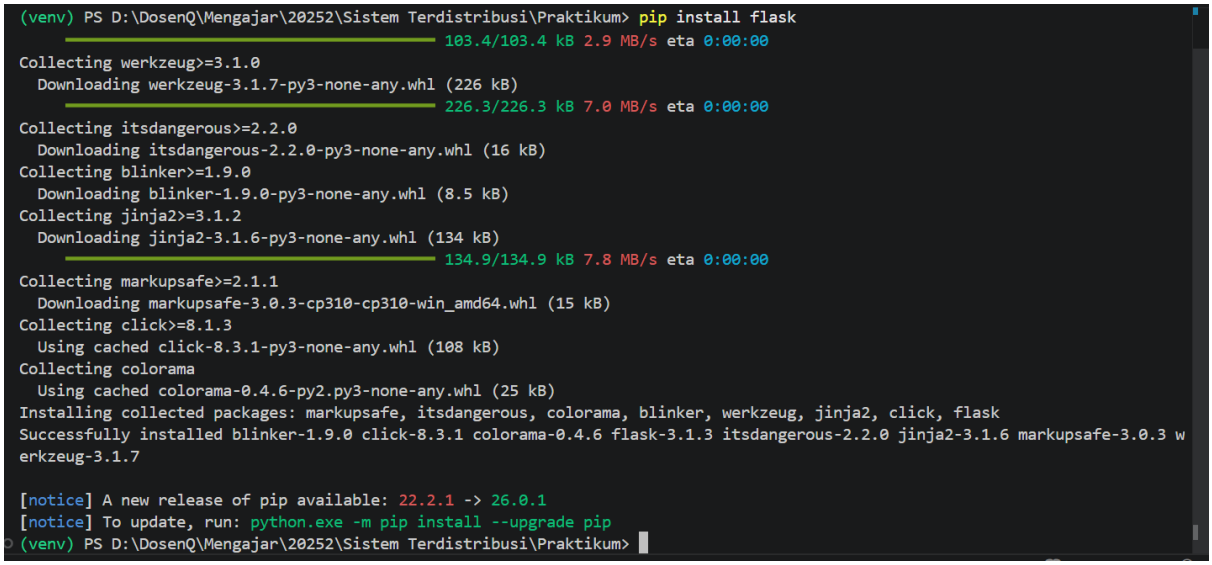
python -m venv venv

3. Aktifkan perintah untuk Windows:

.\venv\Scripts\activate

4. Install Framework Flask

pip install flask



Atau *python -m pip install flask*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> python -m pip install flask
Collecting flask
  Using cached flask-3.1.3-py3-none-any.whl (103 kB)
Collecting werkzeug>=3.1.0
  Using cached werkzeug-3.1.7-py3-none-any.whl (226 kB)
Collecting jinja2>=3.1.2
  Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Collecting blinker>=1.9.0
  Using cached blinker-1.9.0-py3-none-any.whl (
Requirement already satisfied: click>=8.1.3 in c:\laragon\bin\python\python-3.10\lib\site-packages (from flask) (8.3.1)
Collecting markupsafe>=2.1.1
  Using cached markupsafe-3.0.3-cp310-cp310-win_amd64.whl (15 kB)
Collecting itsdangerous>=2.2.0
  Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Requirement already satisfied: colorama in c:\laragon\bin\python\python-3.10\lib\site-packages (from click>=8.1.3->flask
) (0.4.6)
Installing collected packages: markupsafe, itsdangerous, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.7

[notice] A new release of pip available: 22.2.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

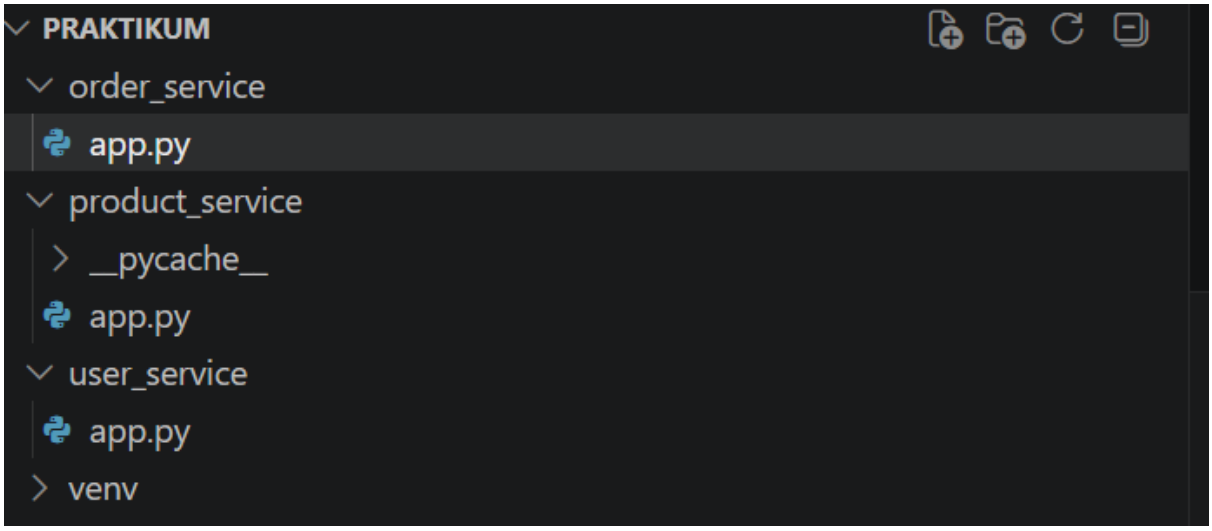
Open folder in new window (ctrl + click)

Cek Flask sudah terinstal apa belum

pip show flask

```
(venv) PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> pip show flask
Name: Flask
Version: 3.1.3
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License:
Location: d:\dosenq\mengajar\20252\sistem terdistribusi\praktikum\venv\lib\site-packages
Requires: blinker, click, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
(venv) PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum>
```

5. Buat File ***app.py*** dimasing-masing sub folder sehingga akan seperti gambar di bawah ini.



1. **Code untuk order_service/app.py**

   Silahkan ketikan code dibawah ini pada file app.py yang berada di folder order_service.

```
from flask import Flask
```

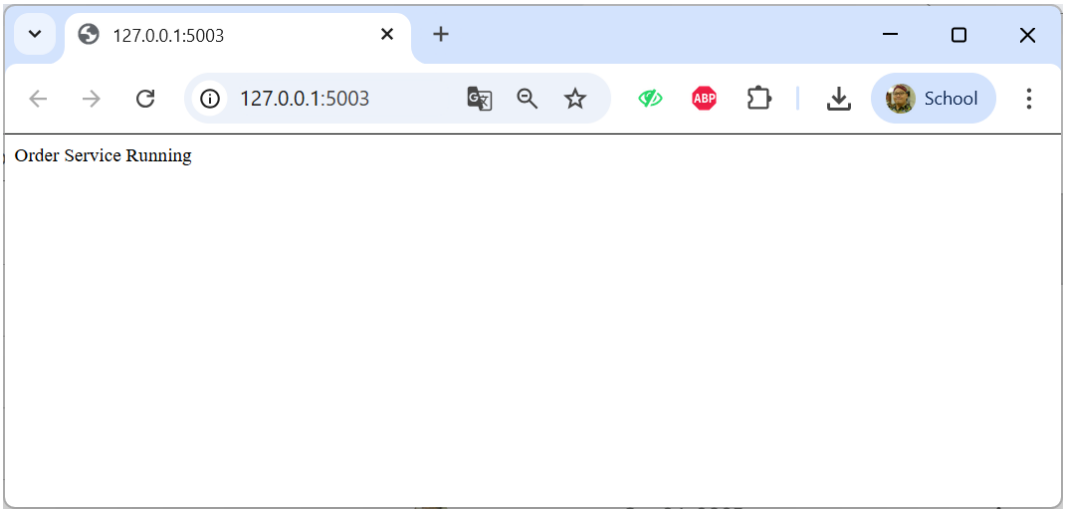
```python
app = Flask(__name__)

@app.route('/')
def home():
    return "Order Service Running"

@app.route('/orders')
def orders():
    return {
        "orders": ["Order1", "Order2"]
    }

if __name__ == '__main__':
    app.run(port=5003)
```

Menjalankan order_service/app.py

setalah itu jalankan dengan perintah sebagai berikut, *python order_service/app.py*. Untuk gambarannya bisa dilihat seperti gambar dibawah ini.

```
(venv) PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> python order_service/app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5003
Press CTRL+C to quit
127.0.0.1 - - [30/Mar/2026 09:39:04] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Mar/2026 09:39:04] "GET /favicon.ico HTTP/1.1" 404 -
```

Sedangkan kita cek di browser seperti gambar dibawah ini.



2. **Code untuk product_service/app.py**

   Silahkan ketikan code dibawah ini pada file app.py yang berada di folder product_service.

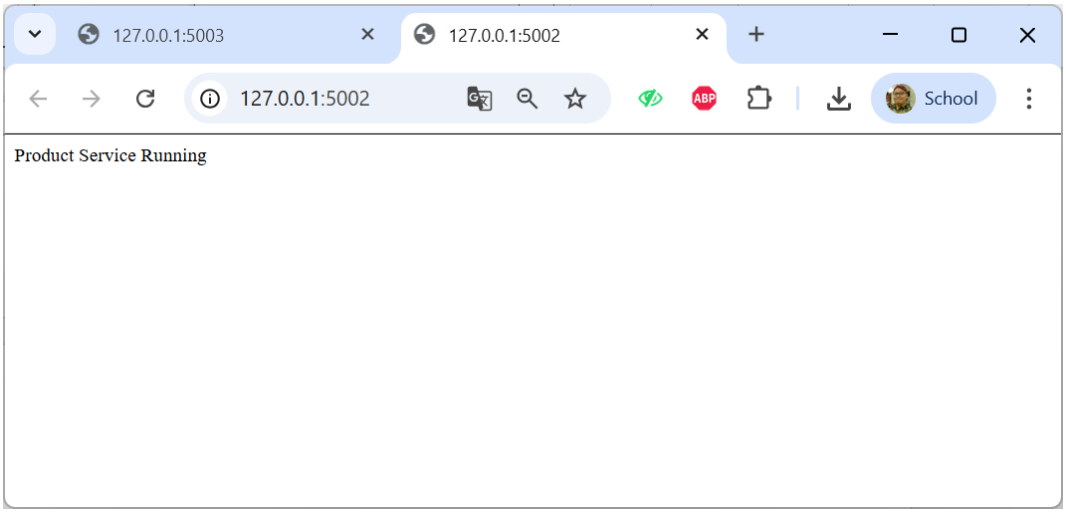
```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "Product Service Running"

@app.route('/products')
def products():
    return {
        "products": ["Laptop", "Mouse", "Keyboard"]
    }

if __name__ == '__main__':
    app.run(port=5002)
```

Menjalankan product_service/app.py

Untuk menjalankan perintahnya sebagai berikut *python product_service/app.py*. untuk tampilannya seperti dibawah ini.

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> python product_service\app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
127.0.0.1 - - [30/Mar/2026 09:46:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Mar/2026 09:46:15] "GET /favicon.ico HTTP/1.1" 404 -
```

sedangkan hasilnya dibrowser seperti gambar dibawah berikut ini.



3. Code untuk user_service/app.py

Silahkan ketikan code dibawah ini pada file app.py yang berada di folder user_service.

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "User Service Running"

@app.route('/users')
def users():
    return {
        "users": ["Admin", "User1", "User2"]
    }

if __name__ == '__main__':
    app.run(port=5001)
```

Menjalankan user_service/app.py

Untuk menjalankan perintahnya sebagai berikut *python user_service/app.py*. Dengan gambaran menjalankan perintahnya seperti dibawah ini.

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> python user_service/app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [30/Mar/2026 09:48:34] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [30/Mar/2026 09:48:34] "GET /favicon.ico HTTP/1.1" 404 -
```

sedangkan hasilnya dapat kita lihat di browser seperti gambar dibawah ini.

Pertemuan 2 – Membuat User Service

Membuat API sederhana untuk user, dengan nama file main.py yang diletakan di *user_service/main.py*. Sedangkan untuk codenya ada di bawah berikut ini.

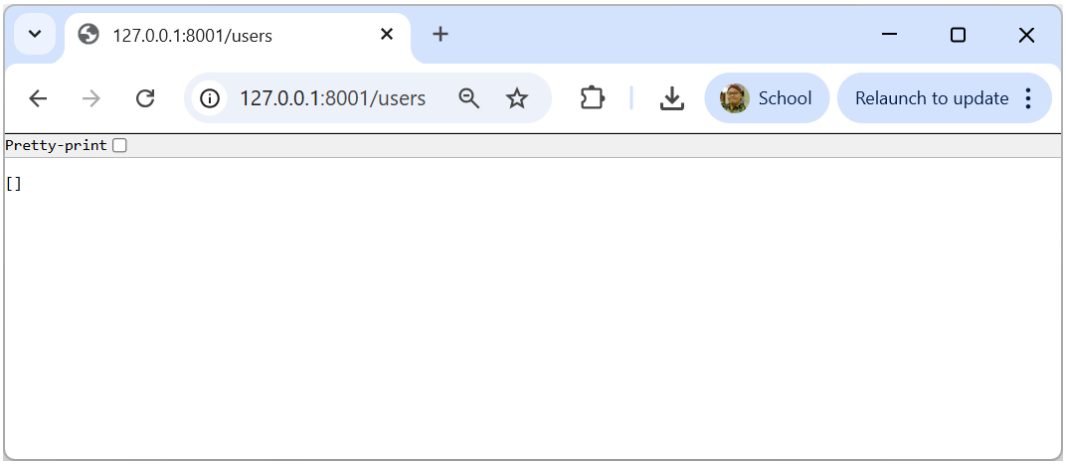
```
from fastapi import FastAPI

app = FastAPI()

# Data dummy user (sementara, belum pakai database)
users = []

# Endpoint untuk menambahkan user
@app.post("/users")
def create_user(name: str):
    user = {"id": len(users)+1, "name": name}
    users.append(user)
    return user

# Endpoint untuk melihat semua user
@app.get("/users")
def get_users():
    return users
```

Menjalankan nya dengan perintah

uvicorn user_service.main:app --port 8001 --reload

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn user_service.main:app --po
rt 8001 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [8796] using StatReload
INFO:     Started server process [30376]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Bisa diakses lewat browser harusnya muncul JSON berupa [], dapat kita lihat pada gambar dibawah ini karena data masih kosong.



Dokumentasi otomatis muncul

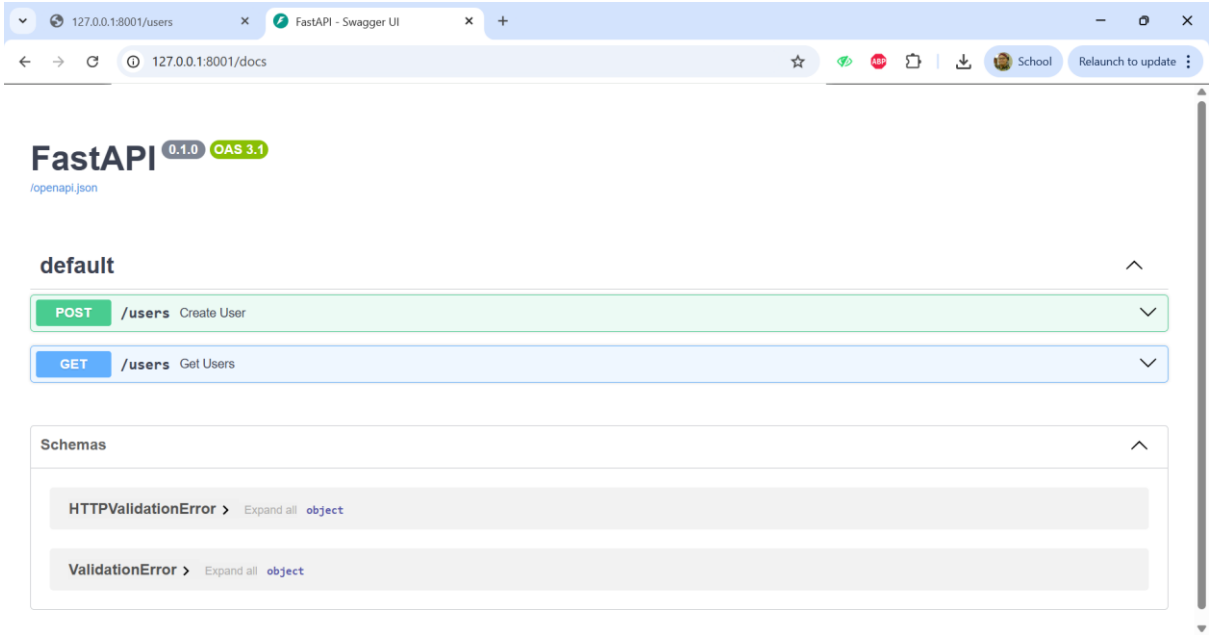
FastAPI punya Swagger UI bawaan buka browser dan ketikan di url

http://127.0.0.1:8001/docs

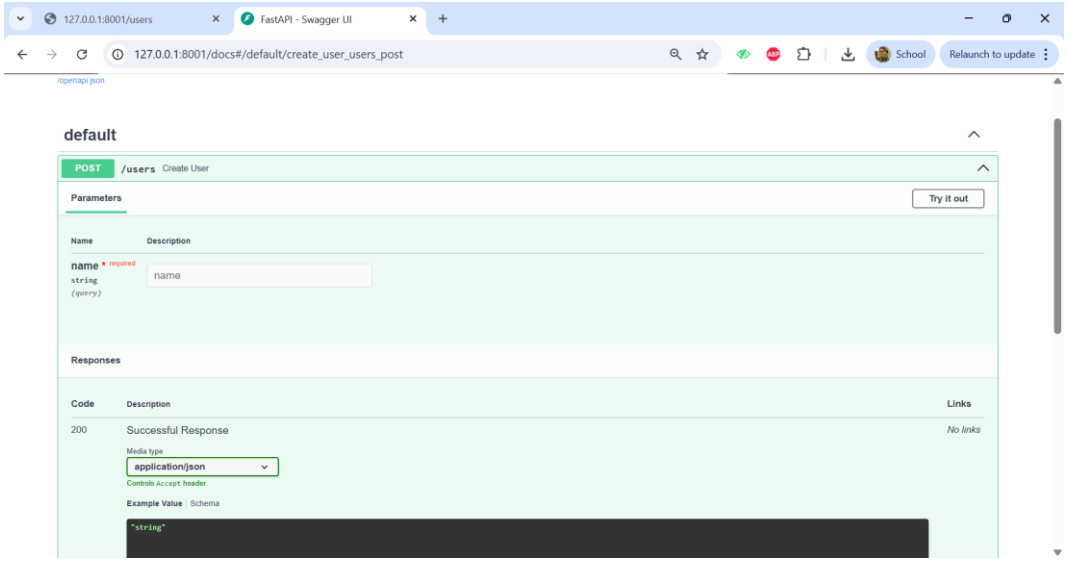
kita akan melihat dua endoint yaitu.

- Endpoint POST /users
- Endpoint GET /users

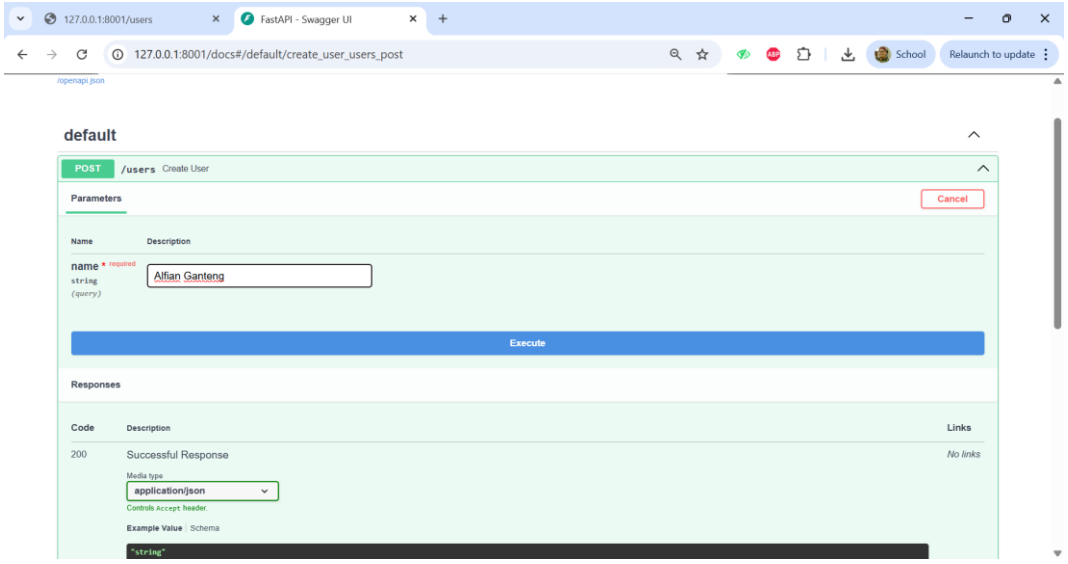
Bisa langsung klik dan mencoba API.



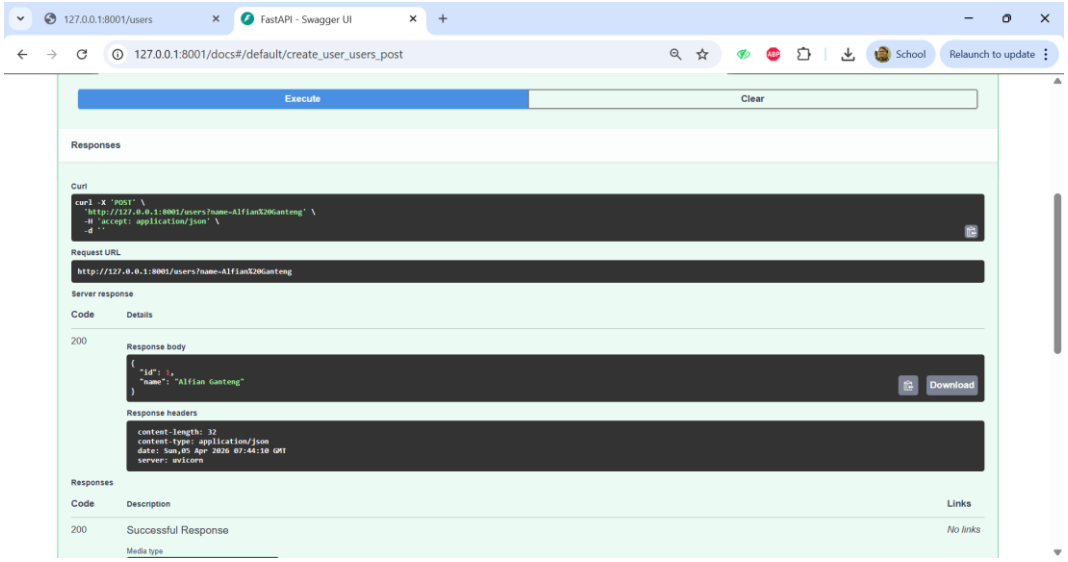
Misalkan kita menambah data user dengan ”POST”.



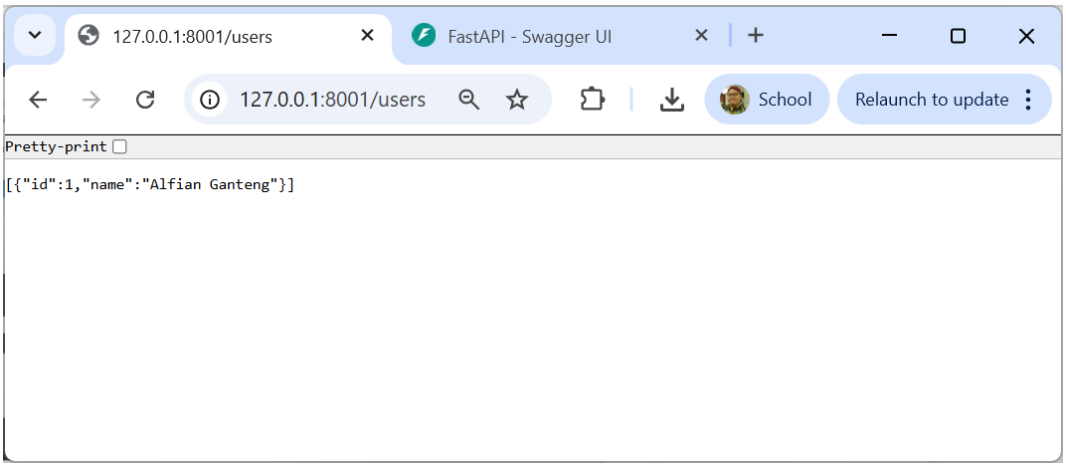
Silahkan klik pada bagian POST, kemudian klik ”Try it out” untuk mengaktifkan inputan pada name. Silahkan inputkan nama kalian atau teman kalian, seperti contoh di bawah berikut ini.



Apabila sudah kita klik ”Execute” untuk menjalankan perintah POST, dan secara langsung data ”JSON” akan tampil seperti gambar dibawah berikut ini.



sedangkan data yang kita coba tadi kita refresh maka data akan muncul seperti yang kita inputkan.



dari sini data belum bisa kita hapus maupun kita edit, karena hanya endpoint ”tambah” saja. Untuk menghilangkan bisa kita reset server api yang kita jalankan terakhir kali tadi. Yaitu dengan menstop kemudian runing kembali dengan perintah *uvicorn user_service.main:app --port 8001 --reload*.

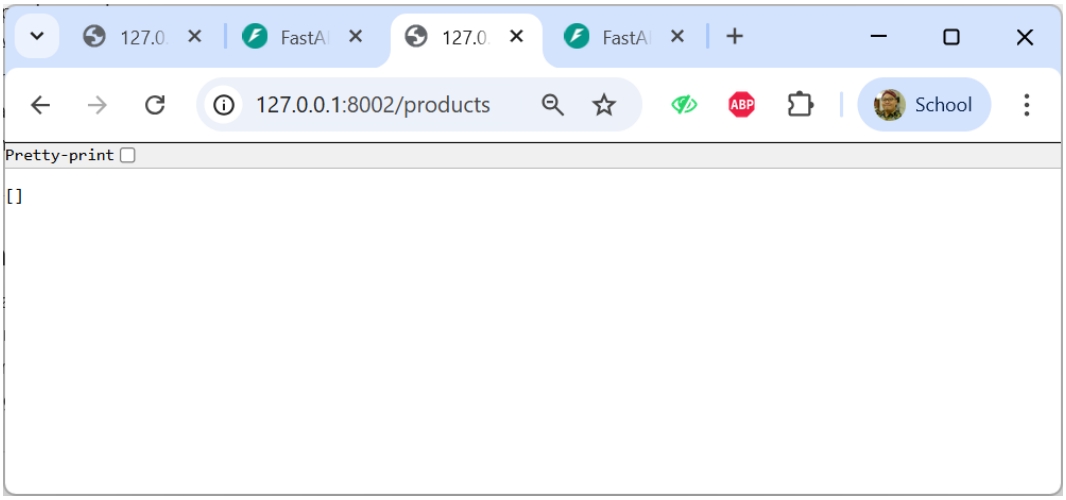
Pertemuan 3 – Product Service

Pertemuan ke 3 hampir sama dengan pertemuan ke 2 yaitu membuat API untuk product. silahkan buat file main.py di *product_service/main.py*, untuk code ada dibawah berikut ini.

```
from fastapi import FastAPI

app = FastAPI()

products = []

# Tambah produk
@app.post("/products")
def create_product(name: str, price: int):
    product = {
        "id": len(products)+1,
        "name": name,
        "price": price
    }
    products.append(product)
    return product

# Ambil semua produk
@app.get("/products")
def get_products():
    return products
```

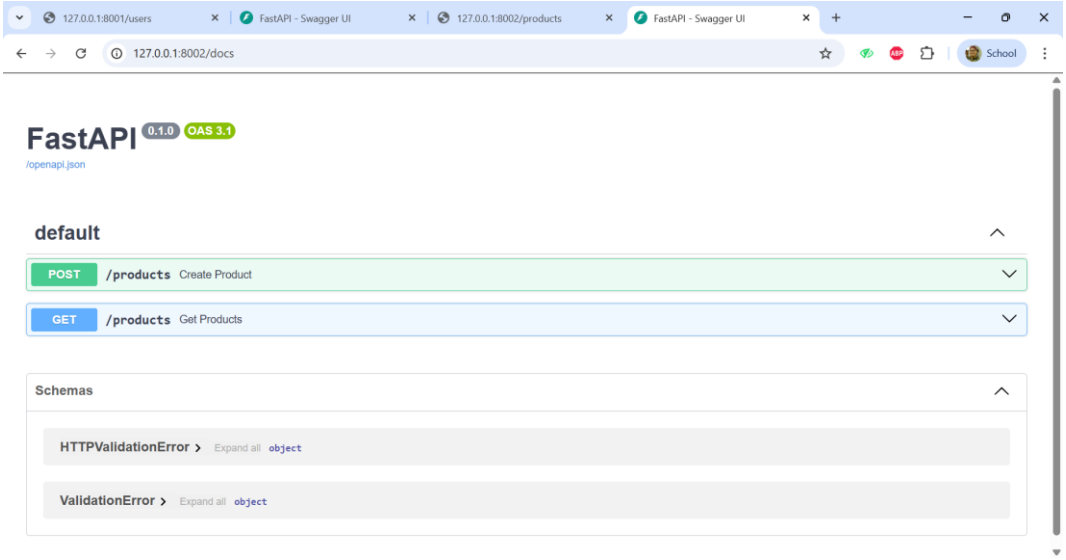
untuk menjalankannya kita gunakan port lain 8002, dengan perintah sebagai berikut *uvicorn product_service.main:app --port 8002 --reload*.

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn product_service.main:app -
-port 8002 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8002 (Press CTRL+C to quit)
INFO:     Started reloader process [27112] using StatReload
INFO:     Started server process [25072]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

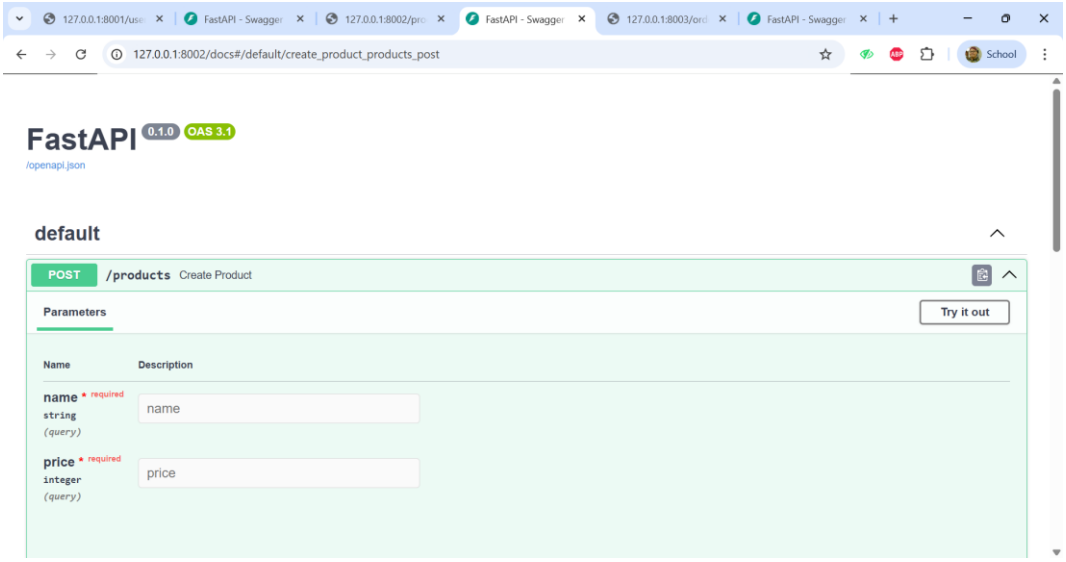


Sama dengan waktu kita cek di browser data product JSON masih kosong, sedangkan gambar dibawah merupakan FastAPI punya Swagger UI bawaan buka browser dan ketikan di url *http://127.0.0.1:8002/docs* sesuaikan port yang dijalankan. Digunakan untuk melihat dua endoint yaitu.

- Endpoint POST /products
- Endpoint GET /products



Untuk cara menambah data dan melihat data sama dengan pertemuan ke 2 saat ***Membuat User Service***.


127.0.0.1:8001/use
FastAPI - Swagger
127.0.0.1:8002/pro
FastAPI - Swagger
127.0.0.1:8003/ord
FastAPI - Swagger
127.0.0.1:8002/docs#/default/create_product_products_post
School
FastAPI
0.1.0
OAS 3.1
/openapi.json
default
POST
/products
Create Product
Parameters
Try it out
Name
Description
name
* required
string
(query)
name
price
* required
integer
(query)
price

Pertemuan 4 – Order Service (Integrasi Awal)

1. Tujuan Kode Order Service (Integrasi Awal) ini dibuat adalah untuk
   a. Menghubungkan beberapa service:
   - User Service (port 8001)
   - Product Service (port 8002)
   - Order Service (service ini)

   dan Ini merupakan adalah konsep dasar microservices.

   b. Membuat relasi data

   Order = kombinasi dari User dan Product, Jadi Order Service tidak menyimpan user dan product sendiri, tapi mengambil dari service lain.

   c. Simulasi sistem nyata

   Sebagai contoh di dunia nyata, Tokopedia / Shopee Saat beli barang ambil data user, ambil data produk di gabungkan jadi order.

2. Code order_service/main.py

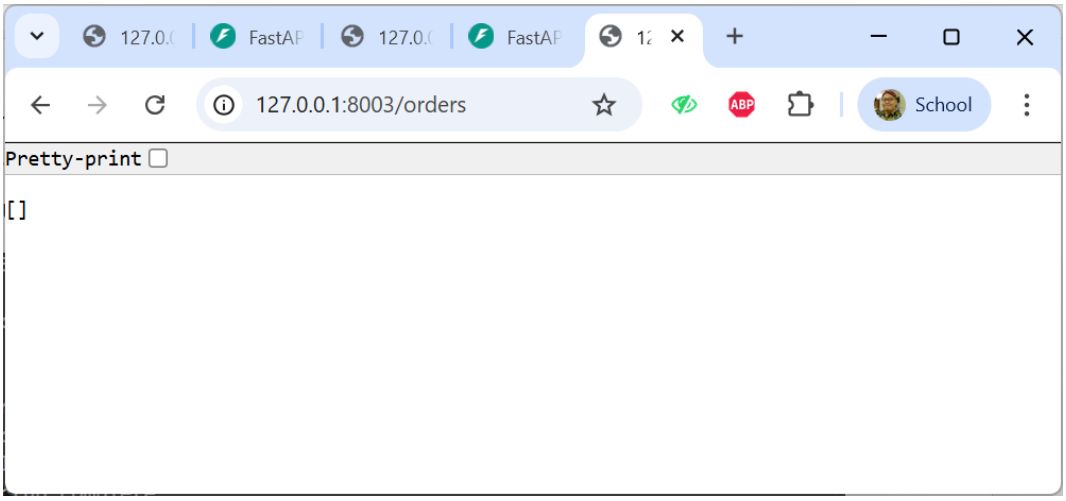
   Membuat code main.py untuk diletakkan di *order_service/main.py*, setelah itu silahkan ketikkan code seperti dibawah ini.

```
from fastapi import FastAPI
import requests

app = FastAPI()

orders = []

# Buat order
@app.post("/orders")
def create_order(user_id: int, product_id: int):

    # Ambil data user dari User Service
    user = requests.get(f"http://localhost:8001/users").json()

    # Ambil data produk dari Product Service
    product = requests.get(f"http://localhost:8002/products").json()

    # Cari user berdasarkan ID
    selected_user = next((u for u in user if u["id"] == user_id), None)

    # Cari produk berdasarkan ID
```

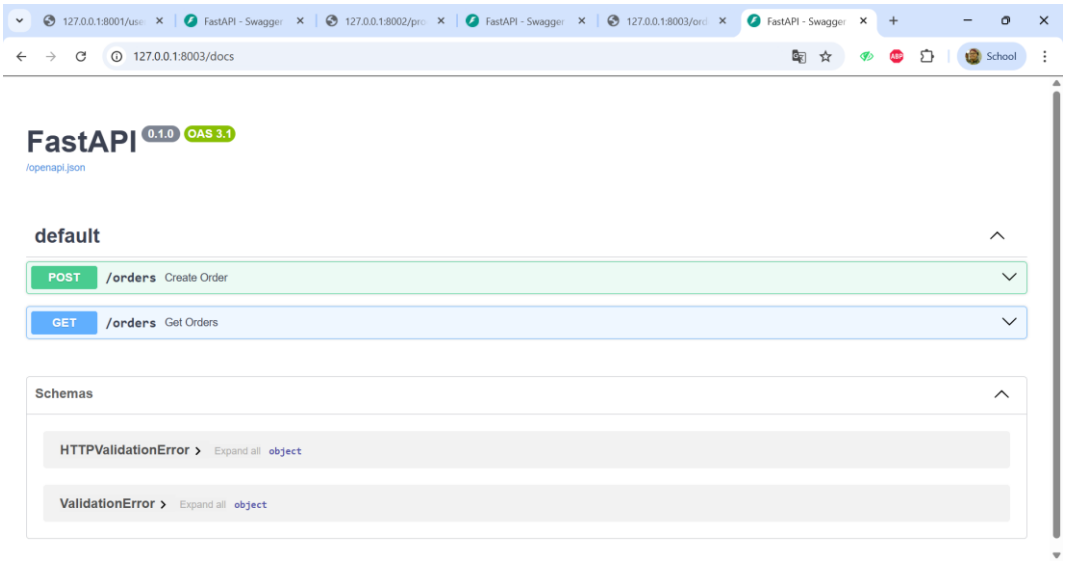
```
    selected_product = next((p for p in product if p["id"] == product_id),
None)

    if not selected_user or not selected_product:
        return {"error": "User atau Product tidak ditemukan"}

    order = {
        "id": len(orders)+1,
        "user": selected_user,
        "product": selected_product
    }

    orders.append(order)
    return order

# Ambil semua order
@app.get("/orders")
def get_orders():
    return orders
```

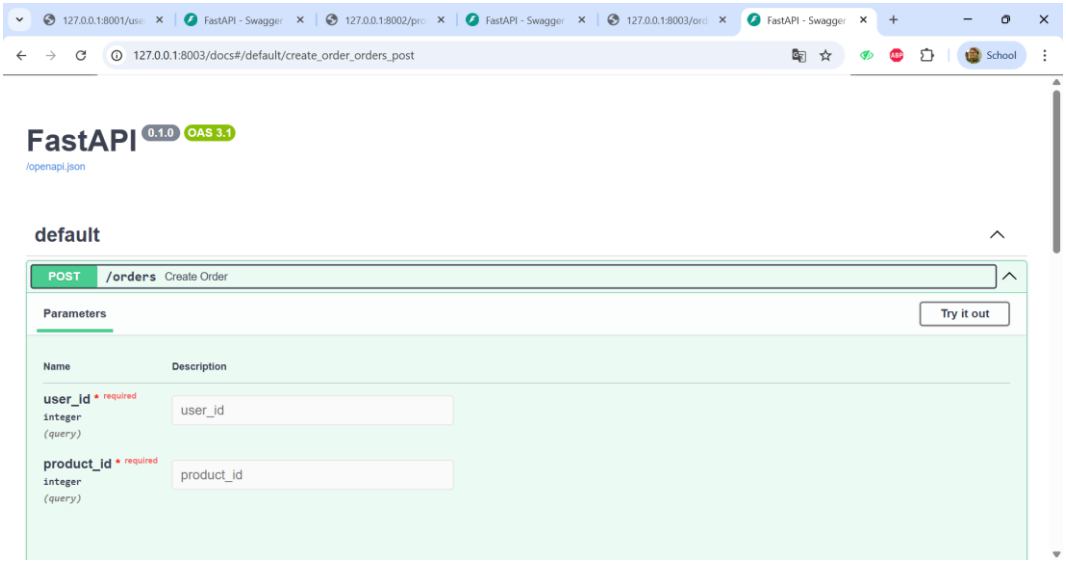
Menjalankan *order_service/main.py* dengan port 8003 untuk perintahnya sebagai berikut *uvicorn order_service.main:app --port 8003 --reload*. Dapat kita lihat pada gambar dibawah ini untuk hasilnya.

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn order_service.main:app --p
ort 8003 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8003 (Press CTRL+C to quit)
INFO:     Started reloader process [3180] using StatReload
INFO:     Started server process [25296]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```



Silahkan buka browser dengan url http://127.0.0.1:8003/orders, kemudian untuk docs API nya http://127.0.0.1:8003/docs. Sama seperti pertemuan 2 dan pertemuan 3 cara menggunakannya, dapat kita lihat gambar dibawah ini.





Pertemuan 5 – API Gateway Sederhana

1. Tujuan API Gateway dibuat untuk
   a. Menyatukan semua service dalam 1 pintu

      Sebelumnya User menggunakan localhost:8001, Product menggunakan localhost:8002 dan Order menggunakan localhost:8003. Mengakibatkan client harus akses banyak URL, sekarang semua lewat Gateway (misalnya localhost:8000).
   b. Menyederhanakan akses client

      Client cukup akses /users, /products dan /orders, tidak perlu tahu service mana di belakangnya
   c. Sebagai "perantara" (proxy)

      Gateway menerima request dari client , kemudian meneruskan ke service yang sesuai selanjutnya mengembalikan response ke client. Ini disebut Request Forwarding / Reverse Proxy sederhana.

2. Code gateway/main.py

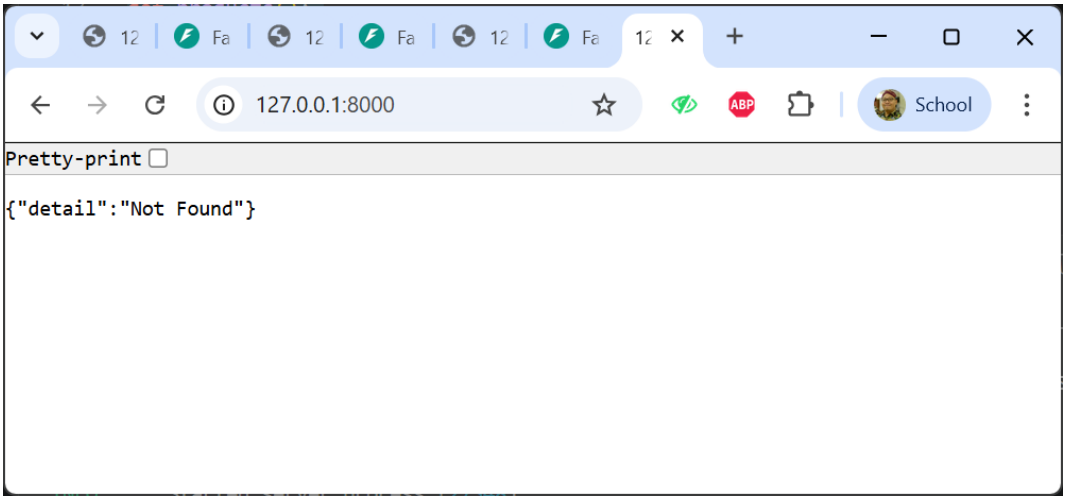
   Silahkan tambahkan folder gateway kemudian baru buat file main.py, sehingga menjadi seperti *gateway/main.py*. Untuk code main.py dapat dilihat seperti dibawah ini.

```
from fastapi import FastAPI
import requests

app = FastAPI()

# Forward request ke user service
@app.get("/users")
def users():
    return requests.get("http://localhost:8001/users").json()

# Forward ke product service
@app.get("/products")
def products():
    return requests.get("http://localhost:8002/products").json()

# Forward ke order service
@app.get("/orders")
def orders():
    return requests.get("http://localhost:8003/orders").json()
```

   Sedangkan untuk menjalankannya dengan perintah sebagai berikut.
   uvicorn gateway.main:app --port 8000 --reload

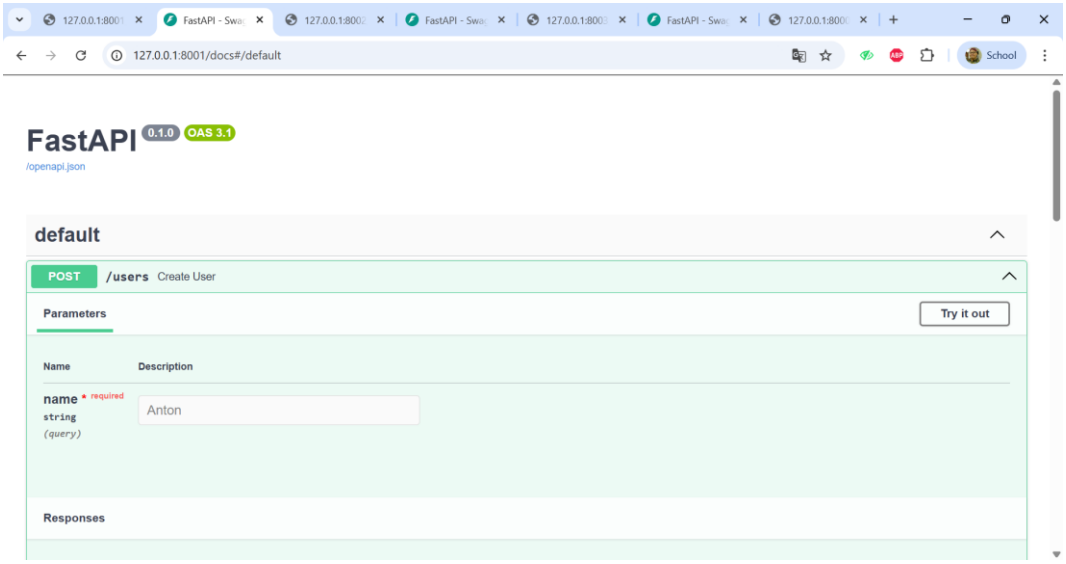
```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn gateway.main:app --port 80
00 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [15708] using StatReload
INFO:     Started server process [27348]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

untuk hasil waktu diakses dibrowser akan seperti gambar dibawah ini.

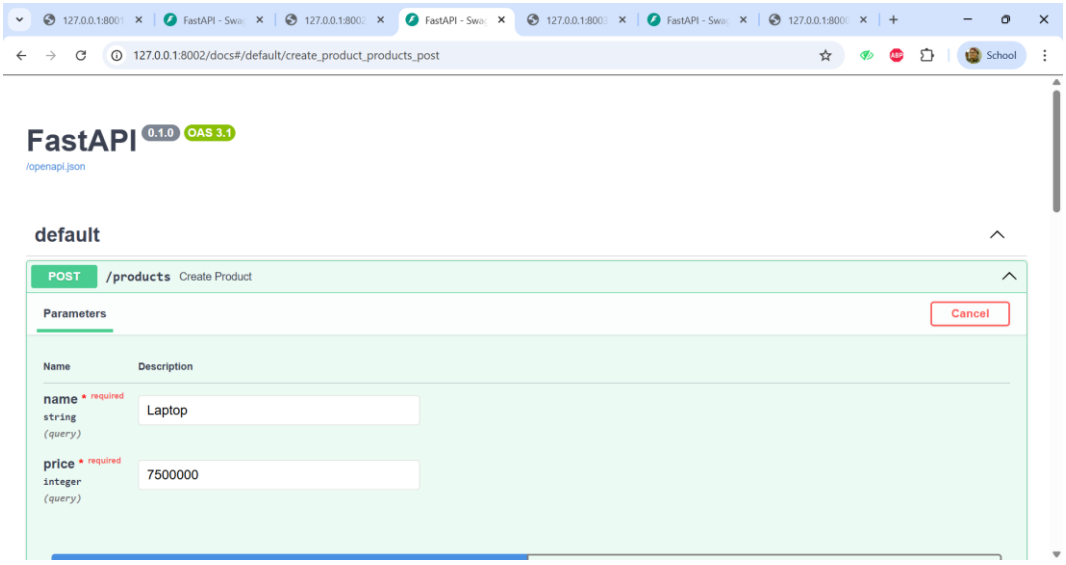


Untuk mengetest gateway http://127.0.0.1:8000/ (akses utama) : Pastikan semua service jalan dan pastikan data sudah di inputkan serta saling berelasi

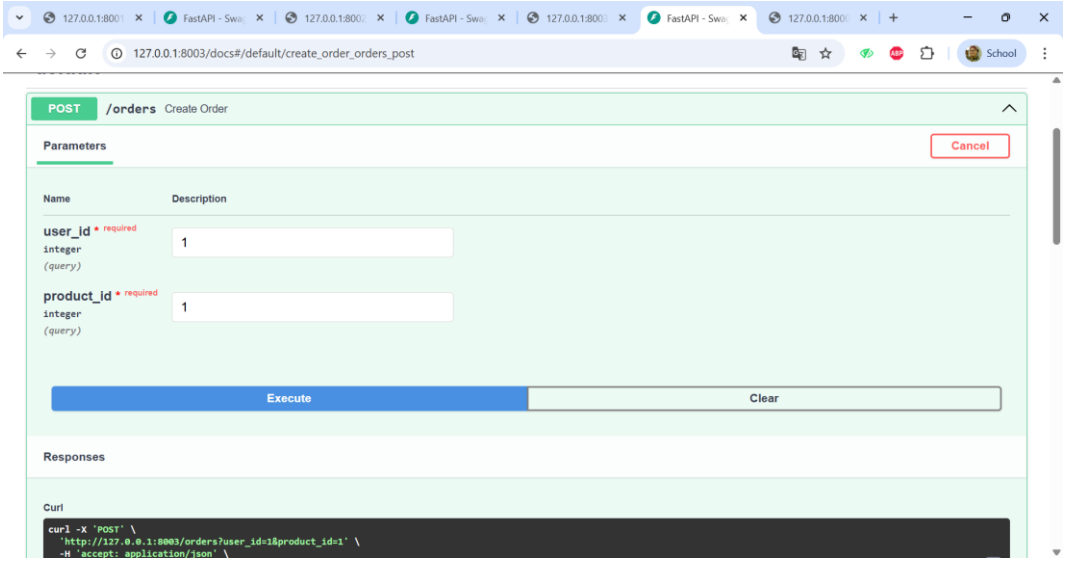
- http://127.0.0.1:8001/users



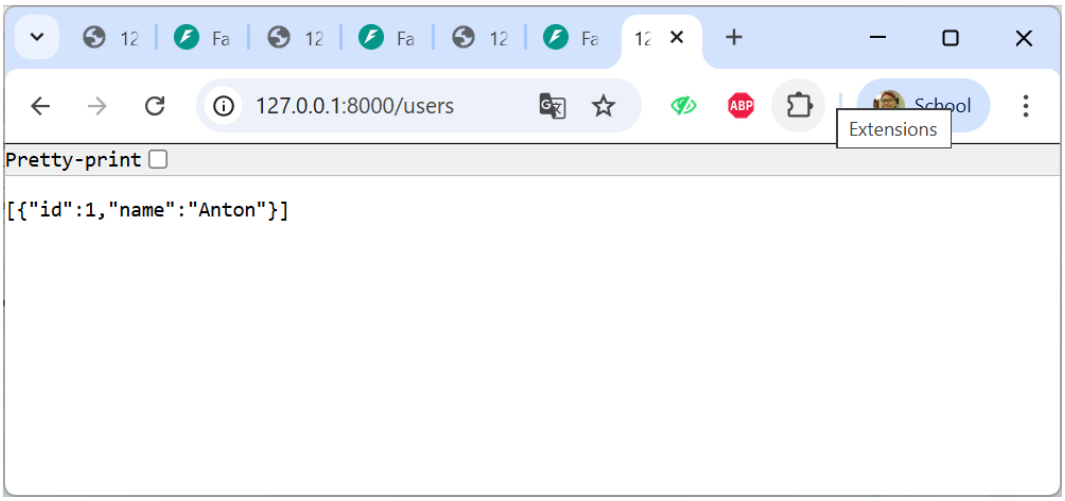
- http://127.0.0.1:8002/products



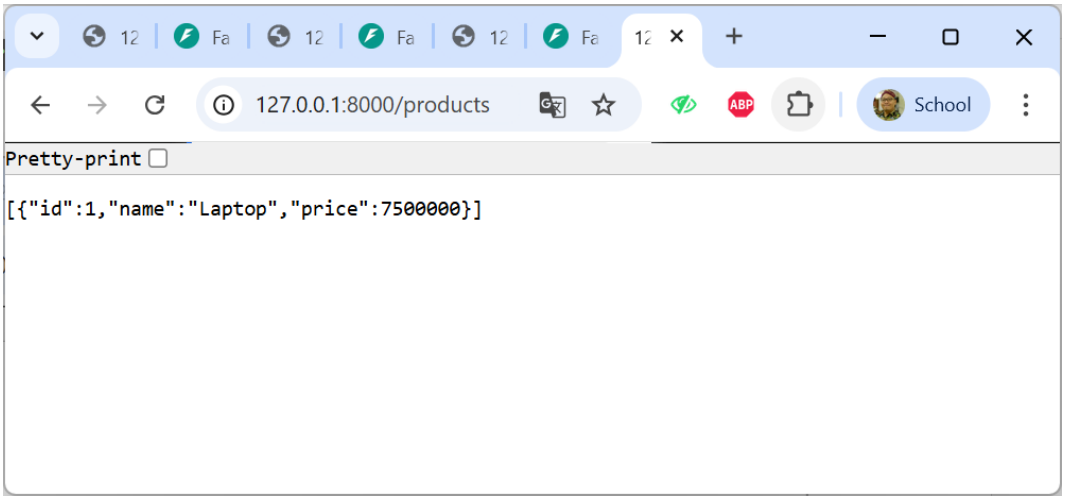
- http://127.0.0.1:8003/orders



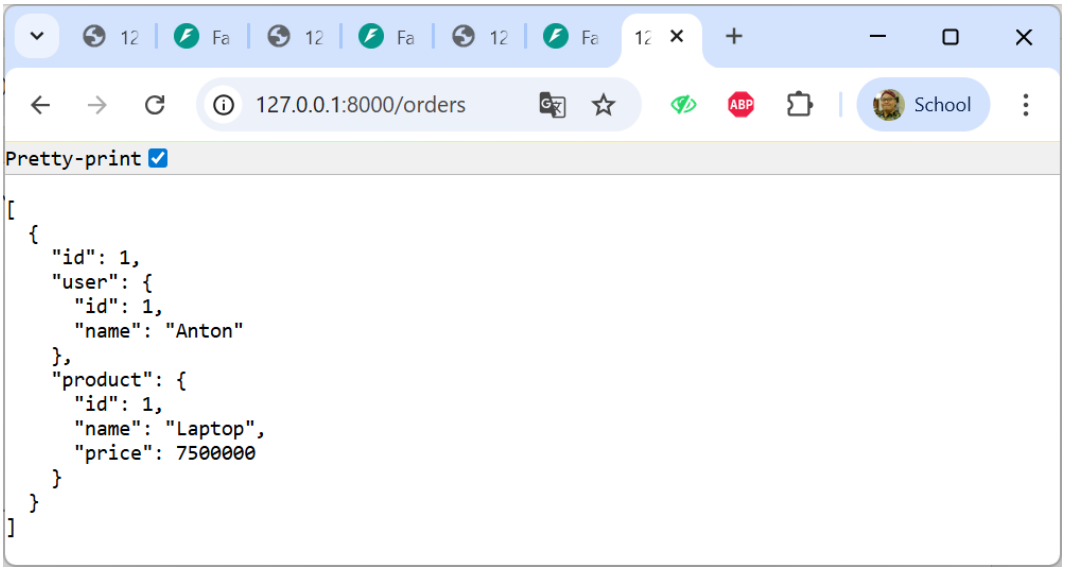
- Selanjutnya test gateway Users



- Gateway Products



- Gateway Orders

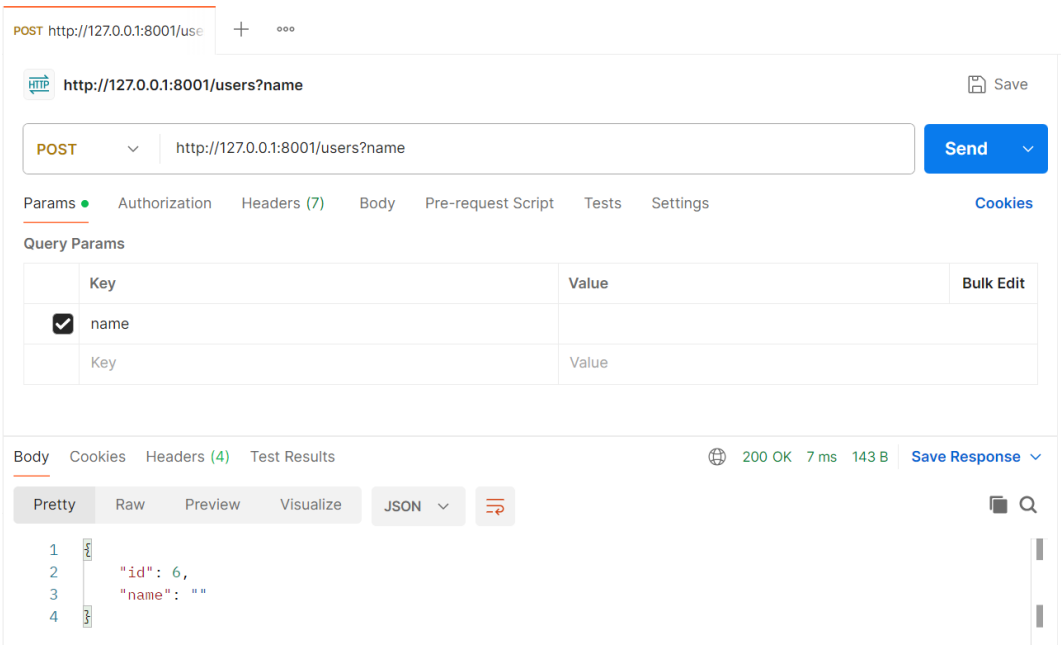



```
[
  {
    "id": 1,
    "user": {
      "id": 1,
      "name": "Anton"
    },
    "product": {
      "id": 1,
      "name": "Laptop",
      "price": 7500000
    }
  }
]
```

Pertemuan 6 – Validasi & Error Handling

a. Tujuan Validasi & Error Handling ini dibuat untuk:
   a. Mencegah data tidak valid masuk, sebagai contoh nama kosong dan inputan tidak sesuai. Tujuan lainnya adalah supaya data tetap bersih dan konsisten
   b. Memberikan respon error yang jelas, daripada error tidak jelas atau crash sebagai contoh code *"raise HTTPException(...)"*. Client langsung tahu apa yang salah dan kenapa gagal
   c. Meningkatkan kualitas API, sehingga API jadi lebih aman, lebih profesional, lebih mudah digunakan client.

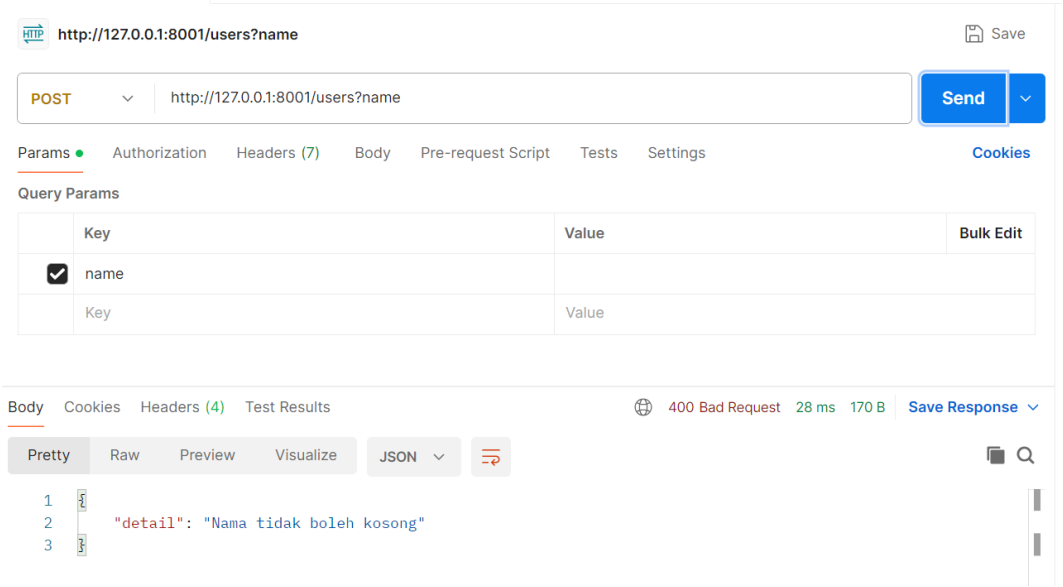
b. Contoh saat kita input data nama, menggunakan software POSTMAN seperti gambar dibawah ini. Saat mengirimkan data kosong maka data akan tersimpan dan bertamah.



Menambahkan code yang di block sehingga menjadi seperti gambar di bawah ini

```
user_service > main.py
1   from fastapi import FastAPI, HTTPException
2
3   app = FastAPI()
4
5   # Data dummy user (sementara, belum pakai database)
6   users = []
7
8   # Endpoint untuk menambahkan user
9   @app.post("/users")
10  def create_user(name: str):
11
12      # Validasi nama tidak boleh kosong
13      if not name:
14          raise HTTPException(status_code=400, detail="Nama tidak boleh kosong")
15
16      user = {"id": len(users)+1, "name": name}
17      users.append(user)
18      return user
19
```

Maka hasilnya setelah kita tambahkan akan menjadi seperti gambar dibawah ini.



Pertemuan 7 – Struktur Lebih Rapi (Refactoring)

1. Tujuan Refactoring (Kenapa perlu dipisah)
   a. sebelumnya (1 file) dan semua jadi satu di main.py. Dimana logic API, Model dan routing dalam 1 file, sehingga sulit dibaca dan dikembangkan
   b. Sesudah (dipisah) user_service/ menjadi beberapa file yaitu main.py digunakan sebagai entry point, file models.py sebagai struktur data dan file routes.py digunakan sebagai endpoint API. sehingga tampilan lebih rapi, scalable, dan profesional.
2. Contoh implementasi untuk folder user_service
   a. models.py

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
```

   b. main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI()
app.include_router(router)
```

   c. routers.py

```
from fastapi import APIRouter, HTTPException
from .models import User

router = APIRouter()

users = []

@router.post("/users")
def create_user(user: User):
    if not user.name:
        raise HTTPException(status_code=400, detail="Nama tidak boleh
kosong")

    new_user = {"id": len(users)+1, "name": user.name}
    users.append(new_user)
    return new_user


@router.get("/users")
def get_users():
```

```
    return users
```

untuk menjalankannya perintahnya juga sama yaitu *"uvicorn user_service.main:app --port 8001 --reload"*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn user_service.main:app --po
rt 8001 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [19016] using StatReload
INFO:     Started server process [27700]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

3. Contoh implementasi Product Service
   a. models.py

```
from pydantic import BaseModel

class Product(BaseModel):
    name: str
```

   b. main.py

```
from fastapi import FastAPI
from .routes import router  # pastikan file routes.py

app = FastAPI()

app.include_router(router)

# endpoint test
@app.get("/")
def home():
    return {"message": "API aktif"}
```

   c. routers.py

```
from fastapi import APIRouter, HTTPException
from .models import Product

router = APIRouter()
products = []

@router.post("/products")
def create_product(product: Product):
    # validasi sederhana
    if not product.name:
```

```
        raise HTTPException(status_code=400, detail="Nama produk tidak
boleh kosong")

    new_product = {
        "id": len(products)+1,
        "name": product.name
    }

    products.append(new_product)
    return new_product

@router.get("/products")
def get_products():
    return products
```

Untuk menjalankannya perintahnya juga sama yaitu *"uvicorn product_service.main:app --port 8002 --reload"*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn product_service.main:app -
-port 8002 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8002 (Press CTRL+C to quit)
INFO:     Started reloader process [28252] using StatReload
INFO:     Started server process [6424]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

4. Contoh implementasi Order Service
   a. models.py

```
from pydantic import BaseModel

class Order(BaseModel):
    user_id: int
    product_id: int
```

   b. main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI()
app.include_router(router)
```

   c. routers.py

```
from fastapi import APIRouter, HTTPException
import requests
from .models import Order

router = APIRouter()
```

```
orders = []

@router.post("/orders")
def create_order(order: Order):
    # Ambil user dan product dari service lain
    try:
        users = requests.get("http://localhost:8001/users",
timeout=5).json()
        products = requests.get("http://localhost:8002/products",
timeout=5).json()
    except:
        raise HTTPException(status_code=500, detail="Service tidak
tersedia")

    selected_user = next((u for u in users if u["id"] == order.user_id),
None)
    selected_product = next((p for p in products if p["id"] ==
order.product_id), None)

    if not selected_user or not selected_product:
        raise HTTPException(status_code=404, detail="User atau Product
tidak ditemukan")

    new_order = {
        "id": len(orders)+1,
        "user": selected_user,
        "product": selected_product
    }
    orders.append(new_order)
    return new_order

@router.get("/orders")
def get_orders():
    return orders
```

Untuk menjalankannya perintahnya juga sama yaitu *"uvicorn order_service.main:app --port 8003 --reload"*

```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn order_service.main:app --p
ort 8003 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8003 (Press CTRL+C to quit)
INFO:     Started reloader process [9224] using StatReload
INFO:     Started server process [31360]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

5. Contoh implementasi Gateway
   a. main.py

```
from fastapi import FastAPI
```

```
from .routes import router

app = FastAPI()
app.include_router(router)
```

b. models.py

```
from pydantic import BaseModel

class Order(BaseModel):
    user_id: int
    product_id: int
```

c. routers.py

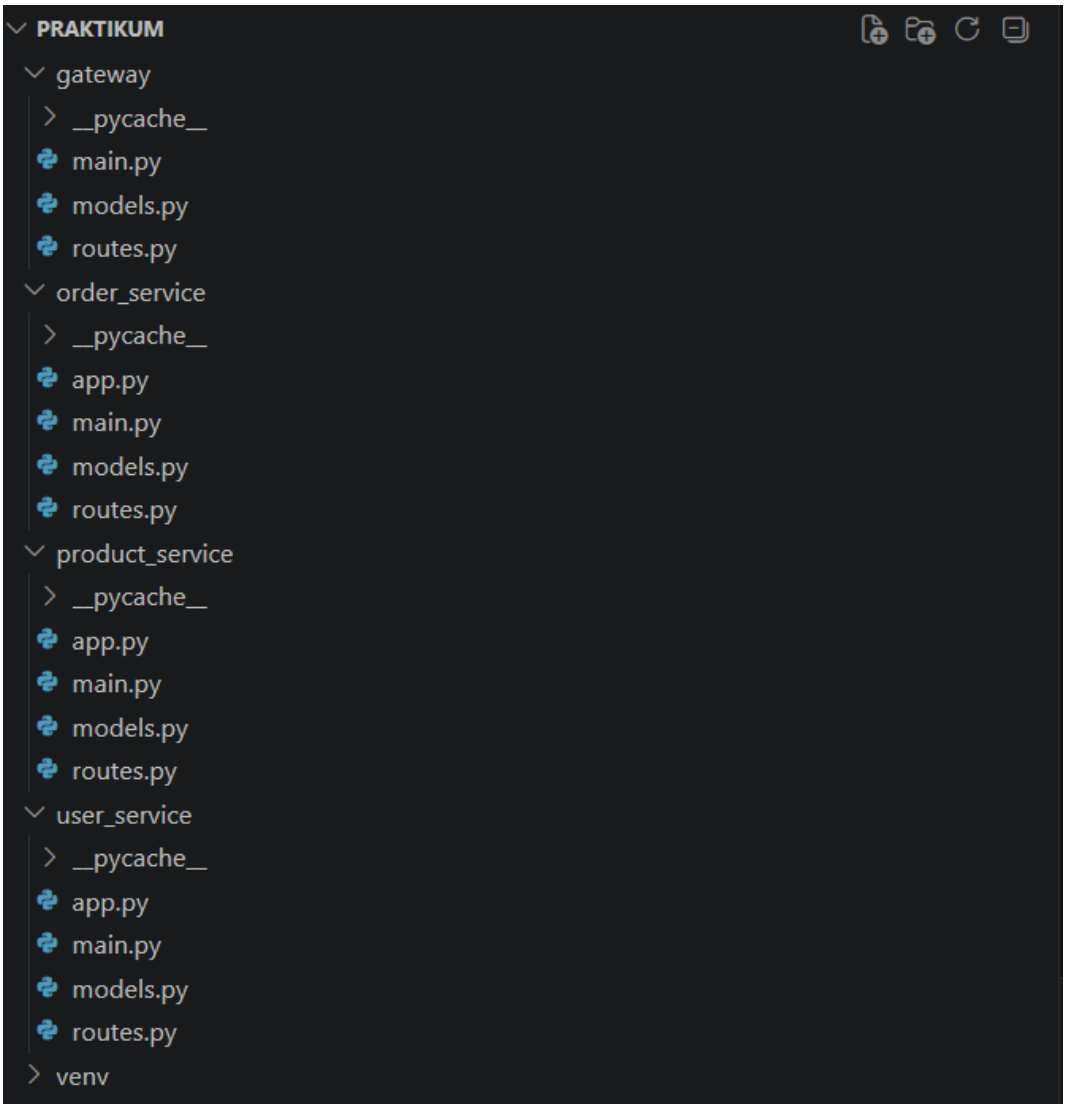
```
from fastapi import APIRouter
import requests

router = APIRouter()

# ====================
# USER SERVICE
# ====================

@router.get("/users")
def get_users():
    return requests.get("http://localhost:8001/users").json()

@router.post("/users")
def create_user(user: dict):
    return requests.post("http://localhost:8001/users", json=user).json()


# ====================
# PRODUCT SERVICE
# ====================

@router.get("/products")
def get_products():
    return requests.get("http://localhost:8002/products").json()

@router.post("/products")
def create_product(product: dict):

    # Simpan hasil request ke variable response
    response = requests.post(
        "http://localhost:8002/products",
        json=product
```

```
    )

    # Return hasil dari product service
    return {
        "status_code": response.status_code,
        "content": response.json()  # gunakan json() bukan text
    }


# ====================
# ORDER SERVICE
# ====================

@router.get("/orders")
def get_orders():
    return requests.get("http://localhost:8003/orders").json()

@router.post("/orders")
def create_order(order: dict):
    return requests.post("http://localhost:8003/orders",
json=order).json()
```

Untuk menjalankannya perintahnya juga sama yaitu ”*uvicorn gateway.main:app --port 8000 --reload*”

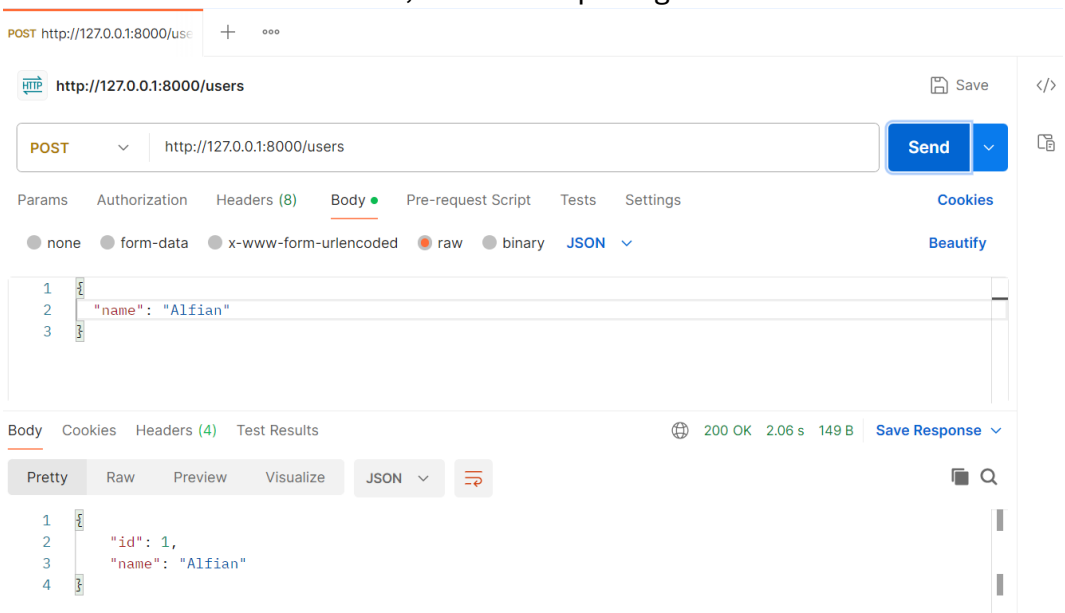
```
PS D:\DosenQ\Mengajar\20252\Sistem Terdistribusi\Praktikum> uvicorn gateway.main:app --port 80
00 --reload
INFO:     Will watch for changes in these directories: ['D:\\DosenQ\\Mengajar\\20252\\Sistem T
erdistribusi\\Praktikum']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [16832] using StatReload
INFO:     Started server process [19908]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

6. Sehingga struktur akhirnya akan seperti gambar dibawah berikut ini.

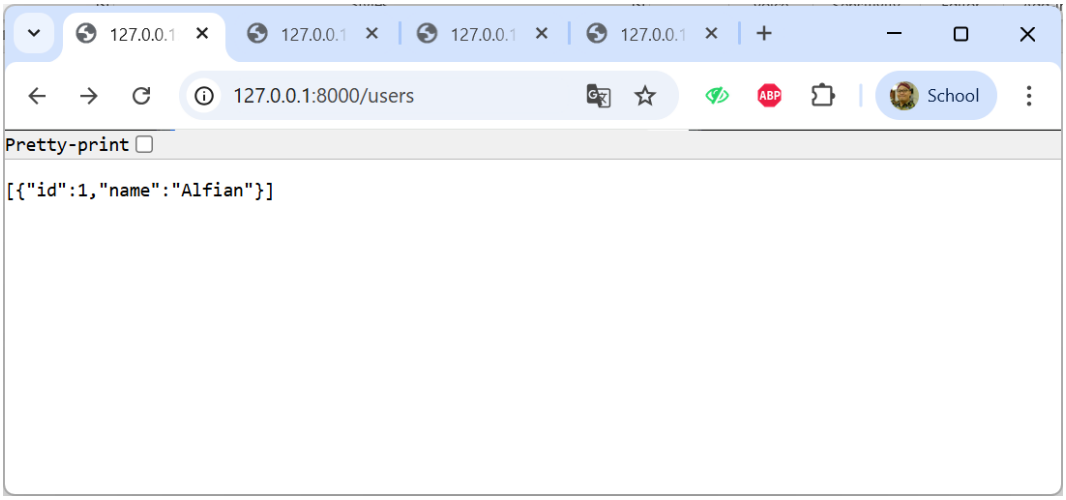


Pertemuan 8 – Final Project & Testing

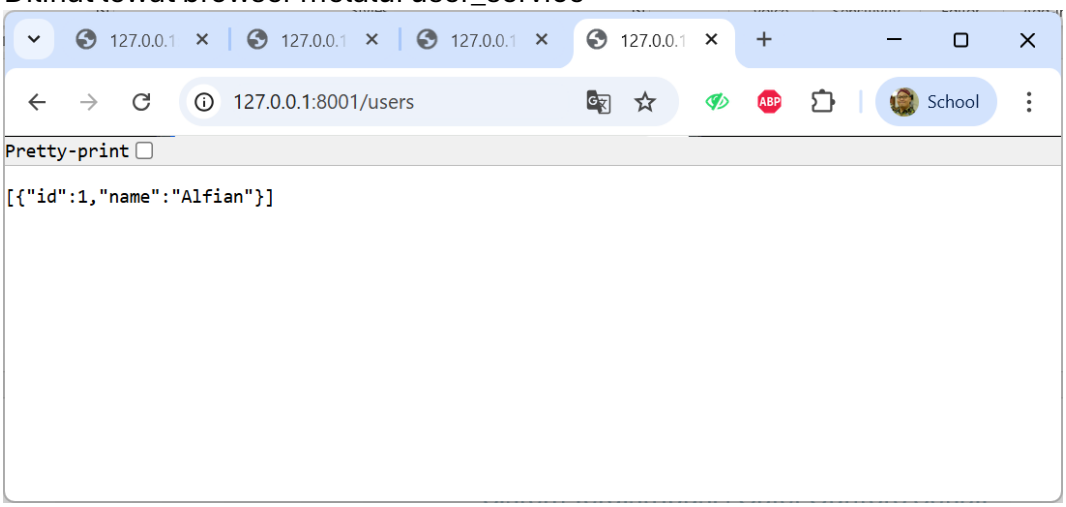
1. Jalankan semua service, dan pastikan berjalan ”startup complete” atau semua aktif.
   a. uvicorn user_service.main:app --port 8001 --reload
   b. uvicorn product_service.main:app --port 8002 --reload
   c. uvicorn order_service.main:app --port 8003 --reload
   d. uvicorn gateway.main:app --port 8000 --reload
2. Gunakan Gateway karena lebih rapi, semua request lewat ***http://127.0.0.1:8000***
3. Testing di Postman (Step by Step)
   Pastikan untuk tester urut dari menambah user, produk hingga order. kenapa harus urut dikarenakan microservice yang kita buat hanya sampel data yang tidak tersimpan jadi hanya digunakan untuk tau relasi data terdistribusi. Kalau sudah paham silahkan mulai mencoba sesuai urutan dibawah berikut ini.
   a. Tambah User
      Gunakan Postman kemudian dengan url http://127.0.0.1:8000/users, menggunakan method ***POST***. Selanjutnya untuk ***Body*** menggunakan ***raw*** dengan format ***JSON*** kemudian ***Send***, bisa dilihat pada gambar dibawah berikut ini.



      Dilihat lewat browser melalui Gateway

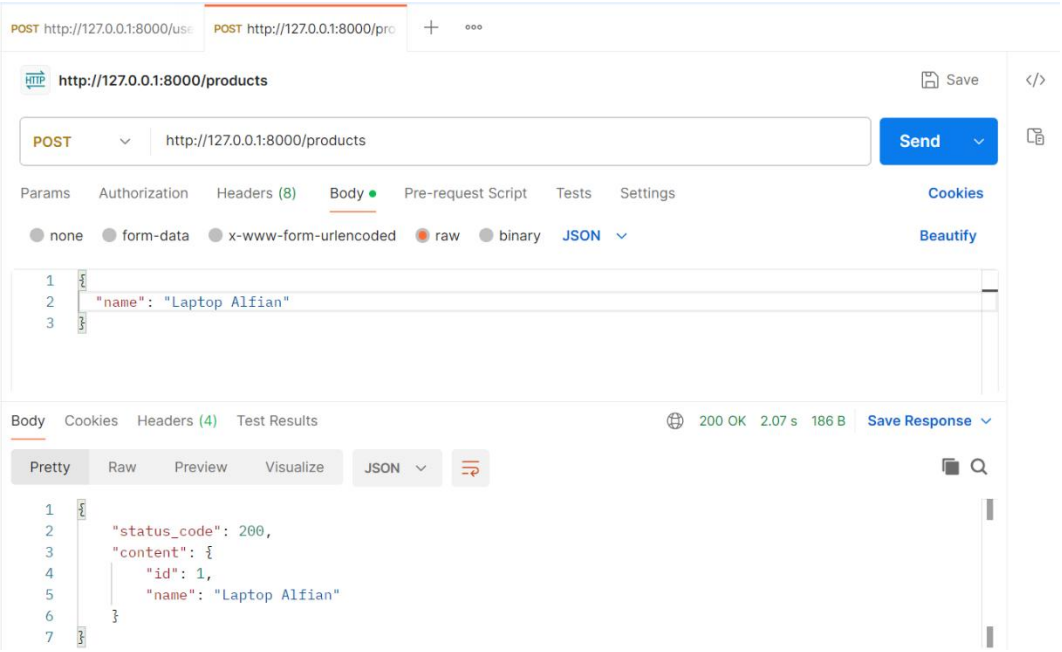


Dilihat lewat browser melalui user_service

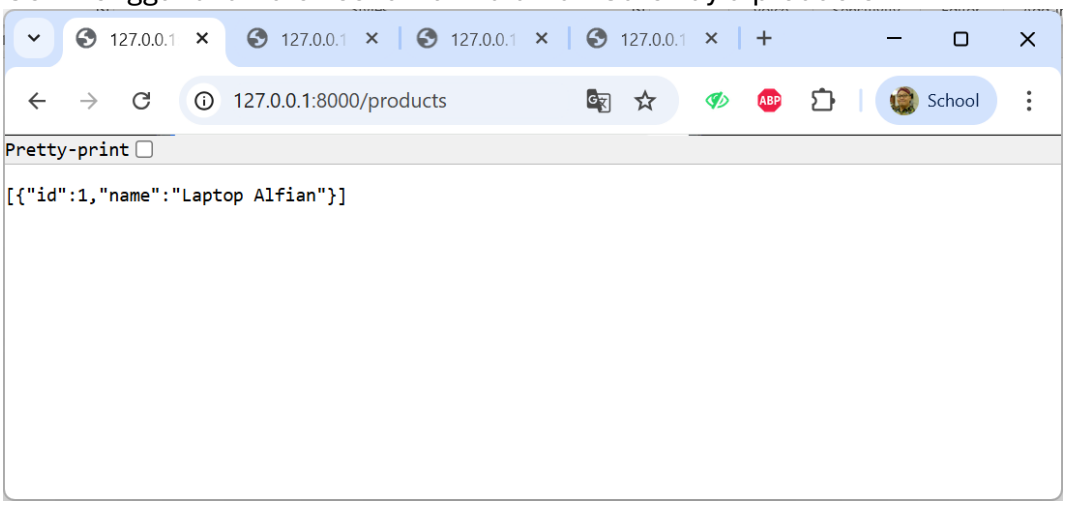


b. Tambah Produk

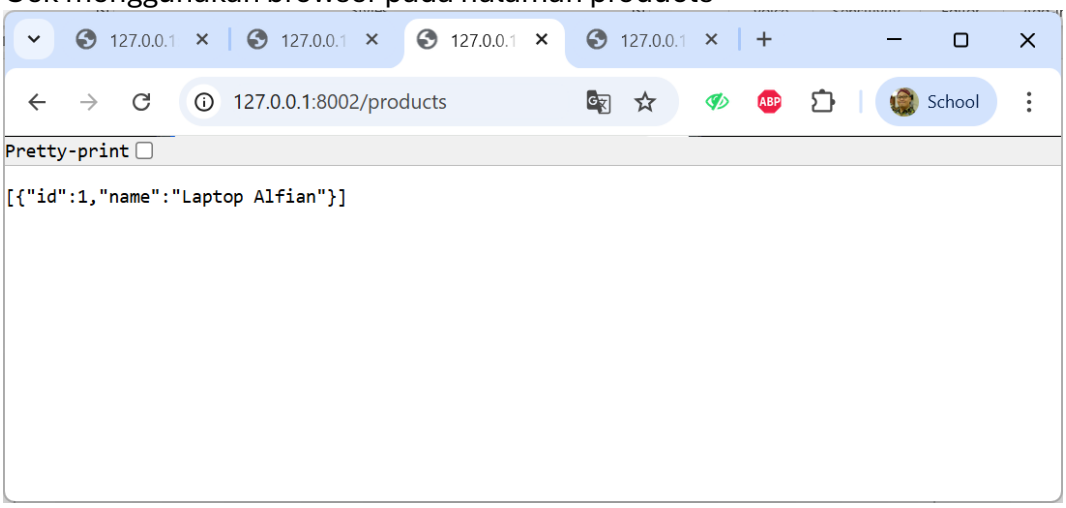
Silahkan ditambahkan product dengan Postman menggunakan method ***POST*** di url http://127.0.0.1:8000/products dan ***Body*** menggunakan ***raw*** serta format ***JSON***, seperti gambar dibawah berikut ini.



Cek menggunakan browser untuk halaman Gateway diproducts

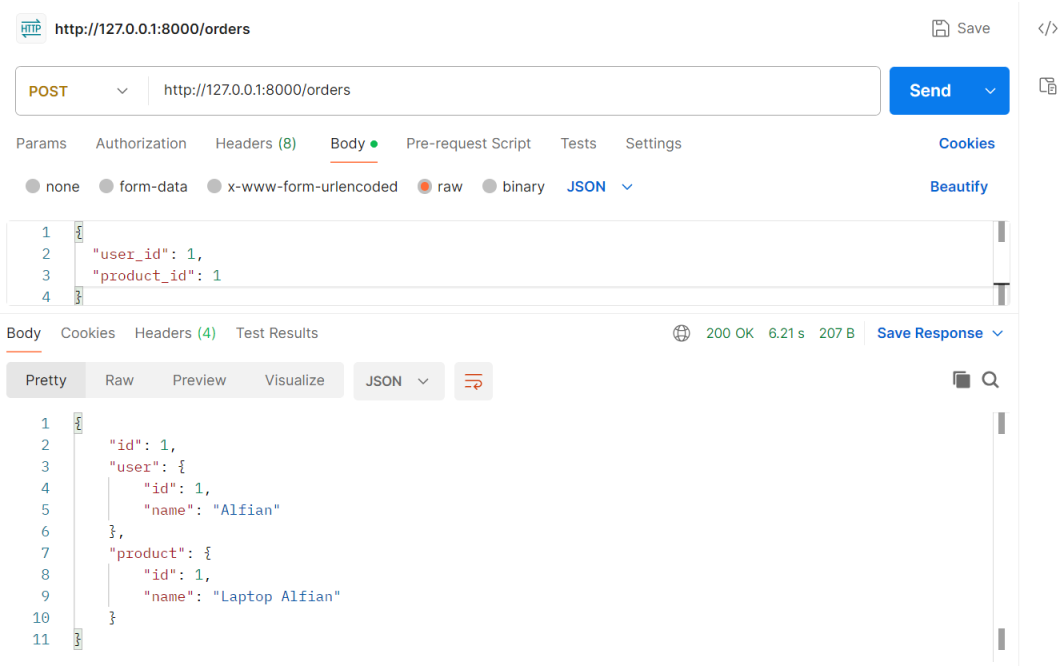


Cek menggunakan browser pada halaman products

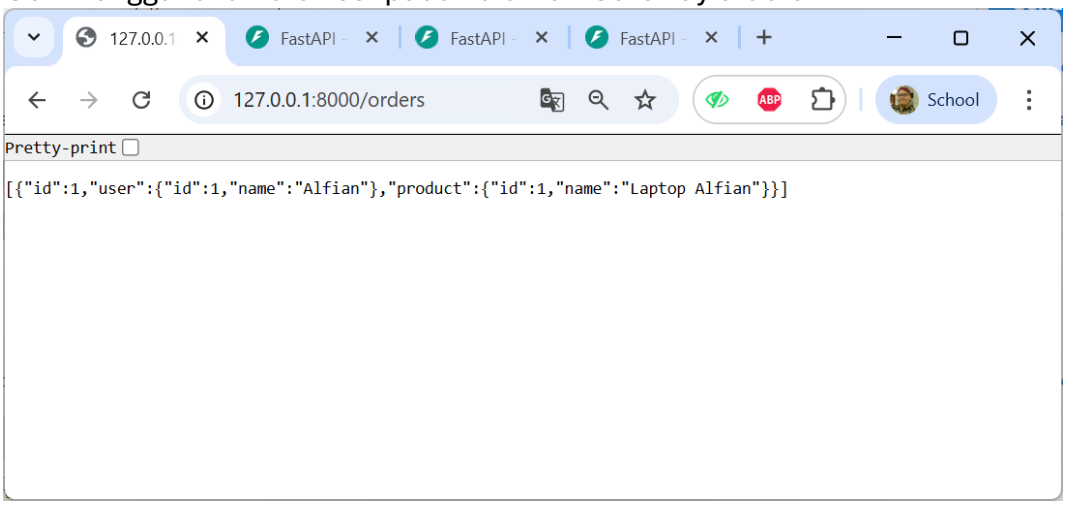


c. Buat Order

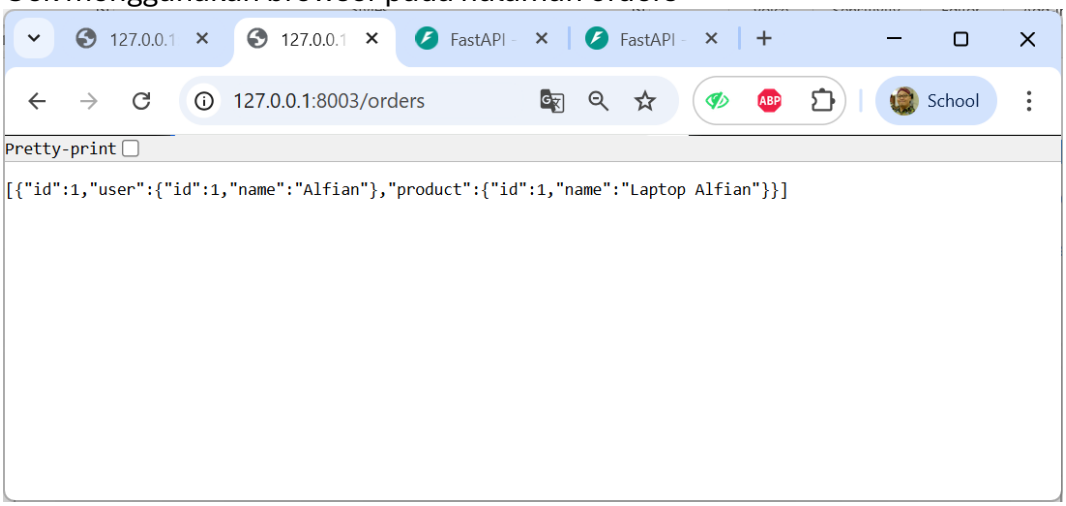
Silahkan ditambahkan product dengan Postman menggunakan method ***POST*** di url http://127.0.0.1:8000/orders dan ***Body*** menggunakan ***raw*** serta format ***JSON***, seperti gambar dibawah berikut ini. Untuk order pastikan user_id dan product_id saling berelasi kalau mengunakan banyak data pastikan data-data yang di inputkan saling berelasi.



Cek menggunakan browser pada halaman Gateway orders



Cek menggunakan browser pada halaman orders



Apabila waktu menjalankan testing tidak ada respon pastikan tiap server berjalan atau runing, biasanya ada salah satu server tidak runing maka waktu proses yang

membutuhkan interaksi terutama waktu gateway mengakses ke masing-masing server, jadi pastikan cek kembali apakah server semua berjalan normal.